Name : S Akshaya

Reg no : 192471061

course code : CSA0618

CIA 1

(i) A) Given,

$R_1 : T(n) : 8T(n/2) + n^2$

$R_2 : T(n) = 16T(n/4) + n^2$

$R_3 : T(n) = 2T(n/2) + n^2$

$R_4 : T(n) = T(n-1) + n$

$R_5 : T(n) = 2^n T(n/2) + n^n$

According to this,

Master time complexity of divicle and conque algorithms.

$$T(n) = aT(n/n) + n^2$$

where.

a = no of algorithms

b = Division factors

f(n) = Dividing & combining

Three cases :

---

conclusion :

$O(n^2)$. Divide & conquer is recommended for large fleet.

5) Given,

$$T(n) = 2T(n/2) + n\log_2 n$$

$$T(1) = 1$$

Result	Lost
0	$n \log n$
1	$n(\log n - 1)$
2	$n(\log n - 2)$
i	$n(\log n - i)$

Total cost :

$$T(n) = n(1 + 2 + \ldots \log n)$$

$$T(n) = O(n \log^2 n)$$

2)

$$T(n) \geq \frac{n\log^2 n}{2}$$

$$\therefore T(n) = O(n \log^2 n)$$

3)

$$T(n) \geq O(n \log^2 n)$$

Pair	Distance
A - B	25
A - C	30
A - D	9
A - E	72
B - C	25
B - D	5
B - E	13
~~C~~ - D	26
C - E	26
~~D - E~~	
D - E	32

closest pair:

$\sqrt{5} \times 10 = 22.36$ ms.

3) square root unnecessary non-negative numbers.

4) comparing distances gives the same closest pair as comparing actual distance

	Ratcliff	Runtime
2,000	1,999,000	99,96,000
4000	7,998,000	3,99,920,000
20,000	99,998,000	

6) $n(n-1) \times 40 \leq 109$

$n(n-1) \leq 50,000,000$

$n \approx 7071.57$

2) $(avg(n) = \frac{n+1}{2}$

$\frac{n}{2} \leq (avg(n) \leq n$

$(avg(n) = O(n)$

3) for fixed p:

$(avg = n(1 - p/2) + p/2$

for $0 \leq p \leq 1$

$\frac{n}{2} \leq (avg \leq n)$

Hence, $O(n)$

∴ final complexity $= O(n)$

---

(4) (1) No. of pairs

A (1, 1), B (4, 5), C (8, 2), D (3, 3), E (7, 7)

formula:

$$P(n) = \frac{n(n-1)}{2} = \frac{5(9)}{2} = 10$$

(2)

formula:

$$a^2 = [x_2 - x_1)^2 + (y_2 - y_1)^2$$

2) Time complexity

Algorithm	Best case	worst case
standard bubble sort	$O(n^2)$	$O(n^2)$
Modified bubble sort	$O(n)$	$O(n^2)$

3) A) (i)

Given array : [18, 42, 7, 25, 61, 9, 33, 50, 14, 27]

comparisons:

Key	comparison
61	5
14	9
99	10

2) $(arg = \frac{n+1}{2} = \frac{11}{2} = 5.5$

3) $avg = p\frac{n+1}{2} + n(1-p)$

4) An extra element placed at the avoids, upward boundary ~~cfousteriorg~~ checking and simplifies the loop.

(ii)

Best case : $O(1)$, worst case : $O(n)$

Average : $O(n)$

∴ cannot apply because n-1 not n/b
$T(n) = 2nT(n/2) + n^n$ and $f(n) = n^n$ is
polynomically bounded.

4.

$T(n) = T(n-1) + n$

$T(n-2) + (n-1) + n$

$T(1) + 2 + 3 + n$

$T(n) = O(n^2)$

5. R_1 standard an optimised first because it as time complexity

conclusion: R_1 takes time large implicts.

☆ 2] Given,

array = [45, 12, 67, 23, 9, 54, 31]

let,

Pass 1 : [12, 45, 23, 54, 31, 67] swaps = 5

Pass 2 : [12, 23, 9, 45, 31, 54, 67] swaps = 3

Pass 3 : [12, 9, 23, 31, 45, 54, 67] swaps = 2

Pass 4 : [9, 12, 23, 31, 45, 54, 67] swaps = 1

Pass 5 : [9, 12, 23, 31, 45, 54, 67] swaps = 0

Time complexity	
Algorithm	Best case
standard bubble sort	$O(n^2)$
modified bubble sort	

complexity

$O(n \log_b a)$

$n \log_b a \cdot \log n)$

$n))$

Pass 6 : [9, 12, 23, 31, 45, 54, 67]

swaps = 0

↳ sorted array.

2) No. of comparisons

$$C(n) = \frac{n(n-1)}{2}$$

$$= \frac{7(6)}{2} = 21$$

Total swaps = 5 + 3 + 2 + 1 = 11

3)

* It evaluatory repeatedly compares adjacent elements

* swap elements it may be in a wrong order
* continues until the array is sorted.

4)

A swap flag is used in stop unnecessary passes.

Algo:

* set swap = 0
* compare adjacent elements.
* If swapping occurs swap = 1
* After, pass check the flag
* If swap = 0, stop the algorithm.

case	condition	complexity
1	$f(n) = O(n \log_b a^{-\epsilon})$	$O(n \log_b a)$
2	$f(n) = O(n \log_b a)$	$O(n \log_b a \cdot \log n)$
3	$f(n) = \Omega(n \log_b a + \epsilon)$	$O(f(n))$

R_1: $T(n) = O(n^3)$

where as R_2:

$$T(n) = O(n^2 \log n)$$

R_3:

$$n \log^2 = n$$

~~n log~~

Hence:

since, n^2 is greater than n, case 3 applies regulatory condition.

$$af(n/n) \leq cf(n)$$

$$2(n/2)^2 = n^2/2$$

3) R_4: $T(n) = T(n-1) + n$